Remaining Useful Life Prediction for Turbofan Engines: Engineered Features and Sequence Learning on C-MAPSS FD001

Kushal Tyagi Enrollment No. 2401010249
Nishant Sharma Enrollment No. 2401010302
Anant Lalwani Enrollment No. 2401020086
Dev Tyagi Enrollment No. 2401010148
Vansh Khod Enrollment No. 2401010493
Newton School of Technology, Rishihood University
Advanced Machine Learning · Project 7, Phases 1 and 2

Abstract—We study remaining useful life (RUL) prediction on NASA C-MAPSS FD001 by comparing a causal degradation-feature pipeline with a sequence-based LSTM. Phase 1 uses 88 engineered features and engine-grouped cross-validation; on the official test set, tuned XGBoost reaches RMSE 10.754, MAE 8.085, and NASA score 161.109. Phase 2 splits training engines before window construction, fits sensor normalization on fit engines only, selects a 50-cycle LSTM using a held-out 20-engine validation set, and refits on all 100 training engines. The validation-selected LSTM obtains RMSE 14.857, MAE 10.559, and NASA score 309.965 on the same 100 official test engines, worse than XGBoost on all three metrics. Its validation RMSE of 8.359 does not generalize to the test population. We report this gap without post-test tuning and retain the engineered-feature XGBoost as the benchmark. The result illustrates that temporal modeling is not inherently superior to a strong, leakage-controlled tabular baseline, especially with a small simulated data set.

Index Terms—predictive maintenance, remaining useful life, C-MAPSS, feature engineering, gradient boosting, LSTM, sequence learning, SHAP

I. Introduction

Unplanned engine failures are expensive and dangerous, while replacing parts on a fixed schedule wastes components that still have useful life. Condition-based or predictive maintenance replaces fixed schedules with decisions based on the measured health of each asset, and its central quantity is the remaining useful life (RUL): the number of operating cycles left before failure [1].

Data-driven prognostics learns RUL directly from sensor histories of past units instead of from a physical degradation model. The NASA C-MAPSS data set [2], [3] has become the standard benchmark for this task: simulated engines with unknown initial wear degrade until failure while 21 sensors are recorded every cycle.

Project 7 compares two ways of using these sensors: (i) engineered degradation features with classical regression models, and (ii) sequence models that learn directly from raw sensor windows. Our contributions are:

- a leakage-free feature pipeline covering cycle, rolling mean, rolling slope, operating-regime and sensor-health features, with a test that proves the features are causal;
- controlled experiments on the target definition, feature groups, models and hyperparameters, all using engine-level cross-validation;
- an evaluation on the official test set with bootstrap confidence intervals, an error breakdown by degradation stage, and a SHAP interpretation;
- a validation-selected, engine-aware LSTM pipeline with window-length and regularization ablations, evaluated on the same official test engines as Phase 1;
- an honest account of the validation-to-test gap and the resulting failure of this LSTM configuration to improve the Phase 1 benchmark.

II. Related Work

Benchmark and early methods. Saxena et al. [2] describe how C-MAPSS simulates damage propagation through an exponential degradation of flow and efficiency in the engine modules, and released the data for the PHM'08 challenge together with an asymmetric scoring function that penalises late predictions. The winning approach of that challenge was a similarity-based method that matches a test trajectory against a library of health-index curves of training engines [4]. Ramasso and Saxena [5] later reviewed more than 70 publications using C-MAPSS and pointed out how inconsistent preprocessing and evaluation make results hard to compare.

Target definition. Heimes [6] observed that degradation is barely visible early in life and proposed a piecewise-linear RUL target that is constant at a maximum value before decreasing linearly. Most later work adopts this with a cap between 120 and 130 cycles.

Feature engineering and sensor selection. Coble and Hines [7] formalised what makes a good prognostic pa-

rameter (monotonicity, prognosability and trendability), which we use to rank sensors. Health indices that fuse several sensors into one degradation signal are a common intermediate step [1], [4].

Machine-learning models. Random forests [8] and gradient boosting [9], in particular XGBoost [10], are strong general-purpose regressors for tabular features. The elastic net [11] combines L1 and L2 penalties and handles groups of correlated predictors, a property that matters for highly collinear engine sensors.

Deep learning. Babu et al. [12] applied convolutional networks to sensor windows, Zheng et al. [13] used LSTMs [14], and Li et al. [15] reached an RMSE of 12.61 on FD001 with a deeper 1-D CNN. These models learn temporal patterns directly, which motivates Phase 2. A recurring question, which this phase addresses, is how much of their advantage remains against a well-engineered feature baseline evaluated under the same protocol.

III. Data Set and Exploratory Analysis

A. The FD001 subset

FD001 contains one operating condition (sea level) and one fault mode (high-pressure compressor degradation). The training set has 100 engines observed until failure (20,631 rows); the test set has 100 engines cut off at a random point before failure (13,096 rows), with the true RUL at the last observed cycle provided separately. Each row holds an engine id, a cycle index, three operational settings and 21 sensor readings. Training lifetimes range from 128 to 362 cycles (mean 206.3, std 46.3), while test engines are observed for 31 to 303 cycles (mean 131.0) and have a true RUL of 7 to 145 cycles. FD001 is well above the 3,000-row requirement of the project.

B. Data quality

We checked for missing, infinite and duplicated values, duplicated (engine, cycle) pairs, engines not starting at cycle 1, and gaps in the cycle sequence. All checks passed for both sets, so no imputation or row removal was needed. Six sensors (s_1, s_5, s_10, s_16, s_18, s_19) and setting_3 are constant, and s_6 takes only two values. The remaining two settings vary only as tiny noise around zero, confirming a single operating regime. We keep the other 14 sensors.

C. Degradation behaviour

Fig. 1 aligns all training engines at their failure point. Most sensors stay roughly constant for a long time and drift monotonically only in the final 100–150 cycles: temperatures (s_2, s_3, s_4), static pressure s_11 and bypass ratio s_15 rise, while HPC outlet pressure s_7, fuel-flow ratio s_12 and the coolant bleeds s_20, s_21 fall. The core speeds s_9 and s_14 behave differently across engines. Cycle-to-cycle noise is large: for s_11 the noise standard deviation is about a tenth of the total change over life, so the trend cannot be seen from consecutive readings.

D. Sensor ranking

Following [7], we computed monotonicity, trendability (mean absolute Spearman correlation with the cycle index) and prognosability on 20-cycle smoothed signals. s_11, s_4 and s_12 rank highest (average scores 0.73, 0.72 and 0.70), the core speeds s_9 and s_14 lowest among the kept sensors (0.49) because engines end at very different values, and the dropped s_6 is worst on monotonicity and trendability. The sensors are strongly collinear: $|r|$ is often above 0.7, and $r = 0.96$ between s_9 and s_14.

E. Implications for the target

To locate the onset of visible degradation, we averaged the sign-aligned z-scores of the 14 sensors into one indicator. Its median across engines is flat until about 200 cycles before failure and reaches 10% of its total rise at about 140 cycles before failure, accelerating thereafter (Fig. 2). Before this point the spread between engines is much larger than the change in the median, so the sensors carry little information about how much life is left. This supports a capped target, which we test formally in Section V-A.

IV. Methodology

A. Problem formulation

For training engine i with last cycle T_i, the RUL at cycle t is $\mathrm{RUL}_{i,t} = T_i - t$. For a test engine, the given RUL at its last observed cycle T_i^{obs} labels every earlier cycle as $\mathrm{RUL}_{i,t} = \mathrm{RUL}_i^{\mathrm{last}} + (T_i^{\mathrm{obs}} - t)$, which allows a per-stage evaluation. Models are trained on the piecewise-linear target

$$y_{i,t} = \min\left(\mathrm{RUL}_{i,t},\, R_{\max}\right), \qquad R_{\max} = 125. \tag{1}$$

B. Feature engineering

Every feature at cycle t uses only cycles $1, \ldots, t$ of the same engine, and every fitted transformation is fitted on training engines only. Inside cross-validation it is refitted on the training engines of each fold.

Operating-regime normalisation. The three settings are standardised and clustered with k-means; each sensor is then z-scored with the statistics of its own regime r:

$$z_{i,t} = \frac{x_{i,t} - \mu_{r(i,t)}}{\sigma_{r(i,t)}}. \tag{2}$$

On FD002 the silhouette score peaks clearly at $k = 6$ (0.997), and normalisation removes the regime jumps that hide the degradation trend (Fig. 3). FD001 has one regime, so the step reduces to a global z-score.

Feature groups. With a short window $w = 10$ and a long window $W = 30$, we compute for each sensor: the normalised reading; its rolling mean and standard deviation over w; its rolling least-squares slope over w and W,

$$\hat{\beta}_t = \frac{\sum_{k=t-w+1}^{t}(k - \bar{k})(z_k - \bar{z})}{\sum_{k=t-w+1}^{t}(k - \bar{k})^2}; \tag{3}$$

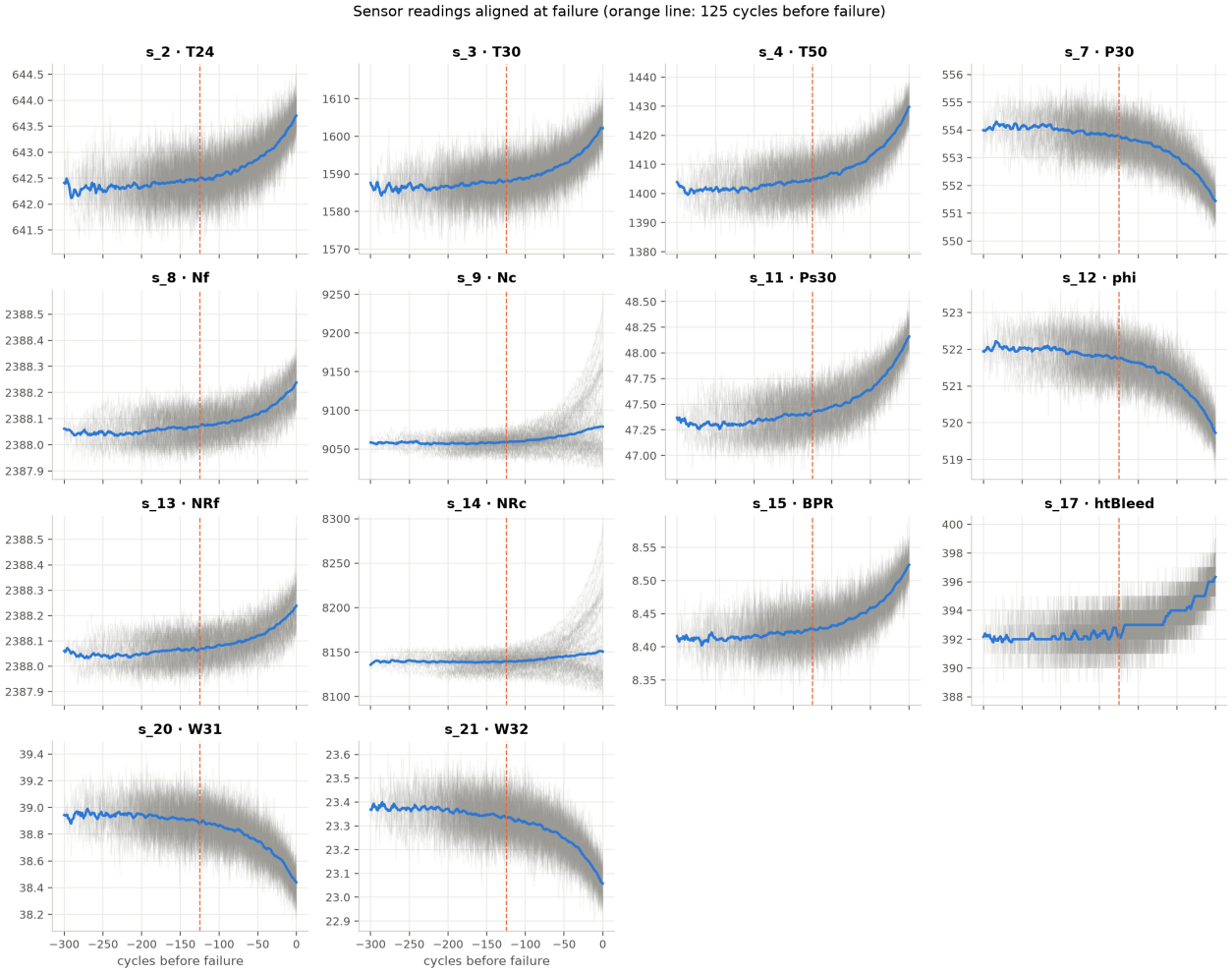


Fig. 1. The 14 retained sensors for all training engines, aligned at failure (grey: individual engines; blue: median). Most sensors are flat early in life and trend over the last 100–150 cycles; the dashed line marks 125 cycles before failure.

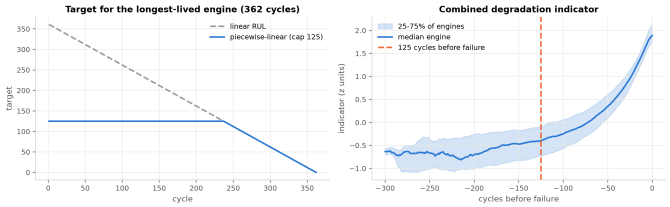


Fig. 2. Left: linear vs. piecewise-linear target. Right: combined degradation indicator aligned at failure (median and interquartile band).

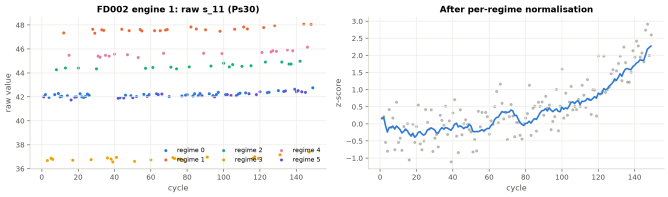


Fig. 3. FD002 engine 1: raw s_11 jumps between six operating regimes (left); after per-regime normalisation the degradation trend becomes visible (right).

and the deviation of the rolling mean from the mean of the engine's first 10 cycles. We also add the current cycle t and a health index: the first principal component of the 14 normalised sensors (64.3% of the variance), sign-aligned to fall with age, together with its rolling mean and W-slope. The result is 88 features with no missing values. Rolling standard deviations and slopes are set to zero until half a window of history exists, instead of being imputed. A truncation test, which recomputes features after deleting all future cycles, gives identical values (maximum difference 0), confirming causality.

C. Models

Elastic Net [11] minimises

$$\frac{1}{2n}\|y - X\beta\|_2^2 + \alpha\left(\rho\|\beta\|_1 + \frac{1-\rho}{2}\|\beta\|_2^2\right) \qquad (4)$$

on standardised features. The L1 term selects features and the L2 term stabilises correlated groups. Random

Forest [8] averages deep regression trees grown on bootstrap samples with random feature subsets, which reduces variance. XGBoost [10] adds shallow trees sequentially, each fitting the gradient of the loss, and minimises a regularised objective $\sum_j \ell(y_j, \hat{y}_j) + \sum_k \left(\gamma T_k + \frac{\lambda}{2}\|w_k\|^2\right)$ over the trees' leaf counts T_k and leaf weights w_k. A mean predictor serves as the baseline.

D. Evaluation protocol

All model selection uses five-fold cross-validation over engine ids (20 engines per fold). Splitting by row would place near-identical consecutive cycles of one engine on both sides and inflate the scores. The official test set is used once, at the end. With $d_j = \hat{y}_j - y_j$ we report RMSE (primary), MAE and the PHM'08 score [2]

$$S = \sum_j \begin{cases} e^{-d_j/13} - 1, & d_j < 0 \\ e^{d_j/10} - 1, & d_j \geq 0, \end{cases} \tag{5}$$

which penalises late predictions (over-estimated RUL) more heavily. Errors are further broken down into early (RUL > 125), mid ($50 <$ RUL ≤ 125) and late (RUL ≤ 50) degradation stages. Hyperparameters are tuned by random search [16] on the cross-validation folds.

E. Phase 2 sequence data and model

The Phase 2 validation split contains 80 fit and 20 validation engines from the 100 run-to-failure training units, selected by engine id with seed 42. The split is made before any overlapping windows are generated. For validation, one deterministic pre-failure cutoff is chosen per held-out engine (seed 43), with true RUL between 1 and 125 where the history permits; this avoids the uninformative all-zero labels that would result from validating only at failure. The official test engines are not used in model selection.

The 14 retained sensor channels are standardized using a StandardScaler fitted only on rows from the fit engines. Validation data use this unchanged transformation. For engine i, window endpoint t, and window length L, the model input is

$$\mathbf{x}_{i,t}^{(L)} = [\mathbf{z}_{i,t-L+1}, \ldots, \mathbf{z}_{i,t}], \tag{6}$$

where $\mathbf{z}$ is the normalized sensor vector. Windows are ordered by cycle and constructed independently inside each engine. If an engine history has fewer than L observations, its first normalized observation is repeated on the left. The endpoint target is $y_{i,t} = \min(\mathrm{RUL}_{i,t}, 125)$.

The selected model is a one-layer LSTM with 14 inputs, hidden size 32, and dropout 0.2, followed by a fully connected regression head:

$$\mathbf{h}_{i,t} = \mathrm{LSTM}(\mathbf{z}_{i,t}, \mathbf{h}_{i,t-1}), \qquad \hat{y}_{i,t} = g(\mathbf{h}_{i,t}). \tag{7}$$

Training uses AdamW, Huber loss, learning rate 0.001, weight decay 10^{-4}, batch size 256, gradient clipping, and seed 42. Checkpoint and epoch selection use validation engines only. The ablation varies $L \in \{20, 30, 50\}$, dropout $\{0, 0.2, 0.4\}$, and weight decay $\{0, 10^{-4}\}$. Selection minimizes validation RMSE against capped RUL. The final model is refitted on all 100 training engines for the selected 32 epochs, with the scaler refit on those training engines; test data are transformed without refitting. Final evaluation predicts once at the last observed cycle of each official test engine.

TABLE I
Effect of the RUL cap (XGBoost, 5-fold CV, scored against true RUL)

Cap	RMSE (RUL≤150)	MAE (RUL≤150)	RMSE (all)
none (linear)	23.42	15.00	36.46
100	18.77	12.76	56.87
115	14.44	10.16	49.52
125	13.17	9.37	45.43
140	13.28	9.30	40.64
155	14.66	10.12	37.18
175	16.91	11.57	34.85

TABLE II
5-fold CV results (capped target, default hyperparameters)

Model	Features	RMSE (mean ± std)	MAE
Mean baseline	–	41.73 ± 0.20	37.01
Elastic Net	raw (14)	21.28 ± 0.54	17.34
	engineered (88)	15.62 ± 1.52	12.69
Random Forest	raw (14)	18.43 ± 0.61	13.56
	engineered (88)	10.48 ± 1.13	7.03
XGBoost	raw (14)	18.57 ± 0.53	13.49
	engineered (88)	9.48 ± 0.62	6.30

V. Experiments and Results

A. Target design

We trained XGBoost on all features with a linear target and with caps from 100 to 175, and scored every variant against the true RUL on validation cycles with RUL $\leq$ 150, the region where maintenance decisions depend on the prediction (Table I). The linear target is clearly worst (RMSE 23.4). Caps of 125 and 140 are practically tied (13.2 and 13.3), and lower or higher caps degrade. We use $R_{\max} = 125$, consistent with the onset analysis and with most published work.

B. Raw sensors versus engineered features

Table II compares every model on the 14 raw (normalised) sensors and on the 88 engineered features, with default hyperparameters. Engineered features reduce the cross-validation RMSE by 27% for Elastic Net, 43% for Random Forest and 49% for XGBoost. On raw sensors, Random Forest and XGBoost perform almost identically, so the input representation, not the model, is the bottleneck. Elastic Net stays behind because a linear function cannot express the flat-then-declining target.

TABLE III
Official FD001 test set (last cycle, true RUL)

Model	Variant	RMSE [95% CI]	MAE	Score
Mean	–	43.07 [38.7, 47.5]	35.90	33629
Elastic Net	raw	21.90 [19.0, 24.7]	17.62	1286
	eng.+tuned	16.78 [14.7, 18.7]	13.69	429
Rand. Forest	raw	18.07 [14.9, 20.9]	13.19	829
	eng.+tuned	11.91 [9.9, 13.9]	8.79	197
XGBoost	raw	18.19 [14.9, 21.2]	12.93	963
	eng.+tuned	10.75 [9.1, 12.4]	8.09	161

TABLE IV
Test RMSE (bias) by stage, all test cycles, true RUL

Model	Early	Mid	Late
XGBoost, raw	73.95	20.87 (+7.2)	28.78 (+17.1)
Elastic Net, eng.	72.57	15.88 (+3.6)	20.34 (+15.9)
Random Forest, eng.	69.39	13.50 (+2.3)	7.46 (+3.3)
XGBoost, eng.	68.38	12.44 (+2.7)	5.70 (+2.0)
n (cycles)	7952	4258	886

C. Feature-group ablation

Fig. 4 shows a forward ablation (adding one group at a time) and a leave-one-group-out ablation. For XGBoost, adding the cycle count lowers RMSE from 18.6 to 16.9, rolling statistics add little, slopes give the largest jump (to 12.0), baseline deviations another large gain (to 9.7), and the health index a final small one (to 9.5). Removing the cycle (+2.1) or the baseline deviations (+1.9) hurts most, while removing slopes changes nothing (0.0) because their information overlaps with the baseline deviations and the health-index slope. The most valuable information is therefore how far and how fast each engine has moved away from its own early-life state, together with its age.

D. Hyperparameter tuning

Random search improved XGBoost from 9.48 to 9.26 ± 0.50 (760 trees of depth 3, learning rate 0.09, minimum child weight 10, subsample 0.67, column sample 0.81) and Elastic Net from 15.62 to 15.41 ($\alpha = 0.60$, $\rho = 0.09$). It left Random Forest unchanged at about 10.5. XGBoost beats Random Forest on all five folds (by 1.25 cycles on average). Tuning contributes about 0.2 cycles, compared with about 9 cycles from feature engineering.

E. Official test set

The models are retrained on all 100 training engines and evaluated on the last cycle of each test engine against the true RUL (Table III). The tuned XGBoost model reaches RMSE 10.75 (95% bootstrap CI 9.1–12.4), MAE 8.09 and $S = 161$. The raw-sensor version scores 18.19 (CI 14.9–21.2), so the improvement is well outside sampling noise. The test RMSE agrees with the cross-validation estimate (9.69 against the capped truth versus 9.26 in CV), showing that the protocol gave a realistic estimate. For context, published FD001 results include an MLP (37.56), SVR (20.96), CNN (18.45) [12], LSTM (16.14) [13] and a deep CNN (12.61) [15]. Evaluation details differ between papers, so this comparison is indicative only, but it shows that the engineered baseline is strong. Fig. 5 shows the predictions.

F. Error by degradation stage

Using all 13,096 test cycles (Table IV, Fig. 6), the engineered XGBoost model is most accurate where it matters most. In the late stage its RMSE is 5.7 with a bias of only +2 cycles, whereas the raw-sensor model has an RMSE of 28.8 and a late bias of +17 cycles, systematically over-estimating the life of failing engines. Elastic Net shows a similar late bias (+16). Mid-stage errors are higher (12.4) because degradation is only beginning there. Early-stage errors against the true RUL are large for every model, but by design: the models are trained to output the cap for healthy engines. Against the capped target, the engineered XGBoost model's early-stage RMSE is only 6.4 (raw: 12.3).

G. Interpretability

TreeSHAP [17] on the final XGBoost model (Fig. 7) attributes most of the predictions to the baseline-deviation group, followed by the health-index features, rolling means and cycle. Raw readings contribute almost nothing once these are available. The single most important feature is the 30-cycle slope of the health index: a fast decline lowers the predicted RUL by up to about 25 cycles. Next come the baseline deviations of s_11 and s_4, the two highest-ranked sensors in Section III-D. A high cycle count raises the prediction. Given the same amount of measured degradation, an engine that took longer to get there degrades slowly, so the model uses age to estimate the degradation rate.

VI. Phase 2 Results

A. Validation ablations

Table V reports the validation-only sweep. A 50-cycle window is the lowest-RMSE candidate on the 20-engine holdout. The XGBoost validation result is measured on the same held-out engines and pre-failure cutoff cycles. These results determine the final LSTM configuration and epoch count; the official test set does not.

B. Official test comparison

After selection, the LSTM was refitted on all 100 training engines for 32 epochs and evaluated once at the final observed cycle of each of the 100 official test engines. Both models use the same test engines, true-RUL labels, and metric implementations. Table VI shows that the LSTM's validation gain does not generalize: its official-test RMSE is 38% higher than XGBoost, and its MAE and asymmetric NASA score are also worse. We did not tune or change the model in response to this test result.

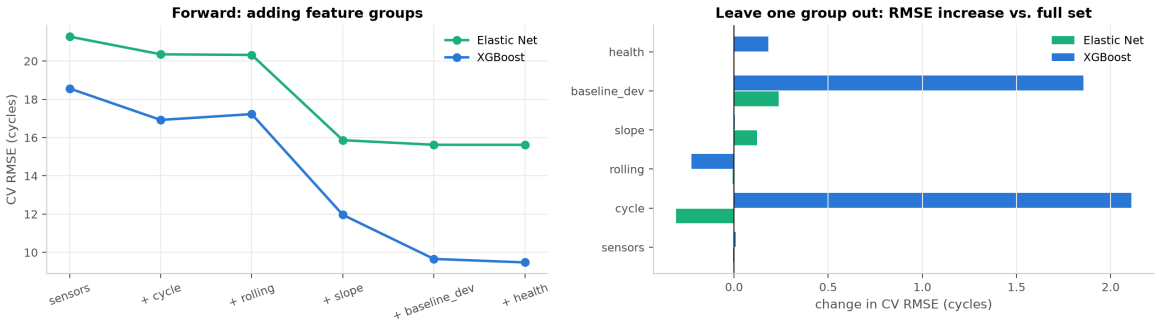


Fig. 4. Feature-group ablation (5-fold CV RMSE). Left: groups added cumulatively. Right: RMSE change when one group is removed from the full set.

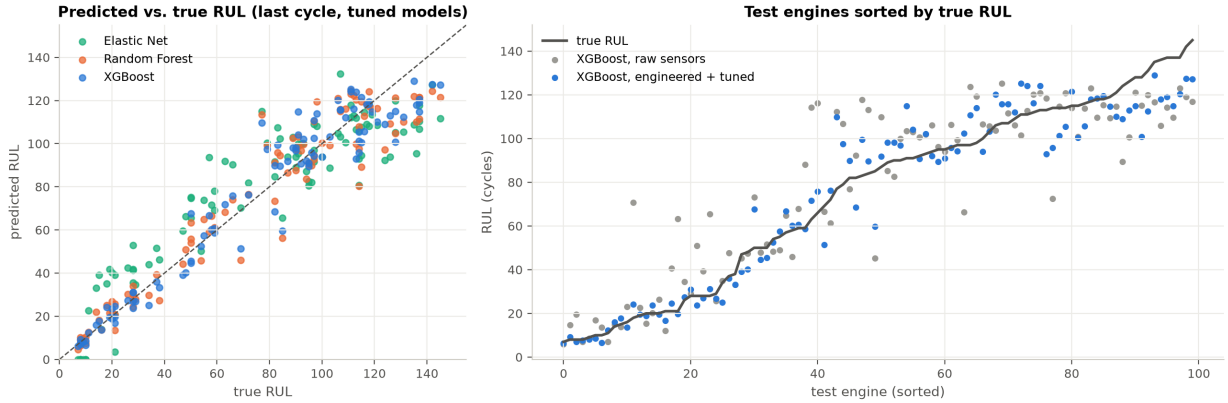


Fig. 5. Test-set predictions at the last observed cycle. Left: predicted vs. true RUL for the tuned models. Right: test engines sorted by true RUL; XGBoost with raw sensors (grey) vs. engineered features (blue).

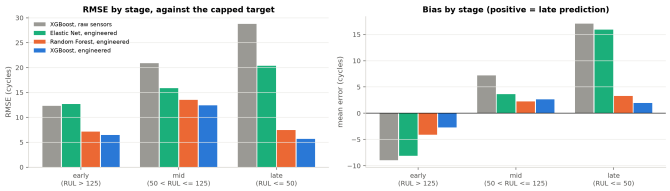


Fig. 6. Test RMSE against the capped target (left) and bias (right) by degradation stage; positive bias means late (dangerous) predictions.

The LSTM RMSE against capped test RUL is 13.412 (MAE 9.489, score 263.105), compared with RMSE 14.857 against uncapped truth. Eleven final test labels exceed 125 cycles. Thus, the target cap explains part, but not all, of the early-life error; capped and uncapped metrics are reported separately and are not interchanged.

C. Final-cycle error by degradation stage

Unlike the Phase 1 stage table, which includes all 13,096 observed test cycles, this comparison contains exactly one final-cycle observation per test engine ($n = 100$). Table VII shows that the LSTM is similar to XGBoost in the late stage but substantially worse in the early and middle groups. The early group is also affected by the training target cap.

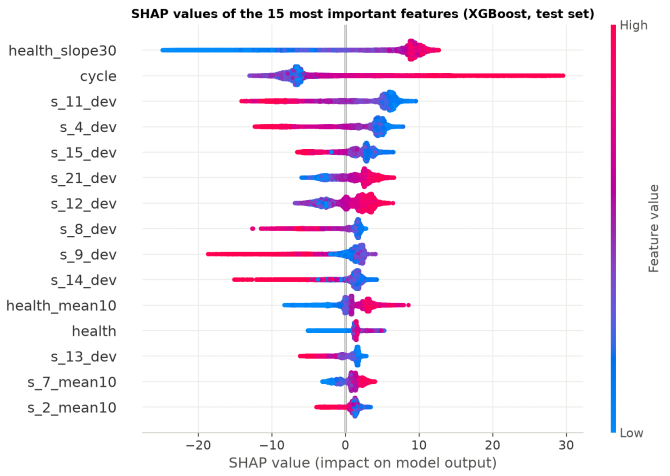


Fig. 7. SHAP values of the 15 most important features (XGBoost, test set). Colour shows the feature value; position shows its effect on the predicted RUL.

TABLE V
Phase 2 engine-holdout validation results (capped RUL)

Candidate	Window	Dropout	Weight decay	RMSE	MAE
XGBoost	–	–	–	10.794	7.521
LSTM	20	0.2	10^{-4}	18.385	12.993
LSTM	30	0.2	10^{-4}	12.895	8.911
LSTM	50	0.2	10^{-4}	8.359	6.521
LSTM	30	0.0	10^{-4}	12.886	9.443
LSTM	30	0.4	10^{-4}	10.082	7.168
LSTM	30	0.2	0	12.090	8.978

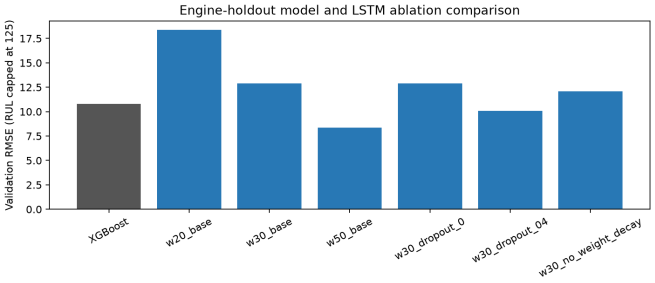


Fig. 8. Validation-only Phase 2 ablations. The 50-cycle window is selected by capped-RUL validation RMSE.

VII. Discussion: Where Fixed Features Fall Short

Using the out-of-fold predictions on training engines, we identified three systematic weaknesses (Fig. 12). First, fast-degrading engines: per-engine error falls as lifetime grows (Spearman $\rho = -0.32$), and engines failing before 160 cycles are predicted late by about 10 cycles on average, against roughly zero bias for the others. The model partly relies on the cycle count as a prior for typical life. Second, the onset of degradation: error peaks for RUL between about 70 and 110, where a 10- or 30-cycle window sees only a short piece of a slow drift. Third, short or unusual histories: test engines observed for fewer than 60 cycles have a higher MAE (10.7 vs. 7.7), and the longest-lived training engine, with a drawn-out decline, is among the worst predicted. These observations motivated Phase 2, but the measured LSTM result does not support a performance improvement: the 50-cycle window was strongest on the validation holdout yet worse on the official test set. This gap is evidence that the single validation split was not sufficiently representative and that a temporal model's capacity does not guarantee better generalization.

Limitations. FD001 is simulated, has one operating condition and one fault mode, and its official test set has only 100 engines. Phase 2 uses a single seeded 20-engine validation holdout and a compact LSTM sweep; the selected validation score did not generalize to test. No Temporal CNN or hybrid model was evaluated. The official test set was used for the final comparison only and was not used to revise model selection. Generalization to FD002–FD004 and real engines remains unverified. Phase 1 stage analysis uses all test-cycle rows, whereas Phase 2 stage results use one terminal observation per engine; these populations are not interchangeable.

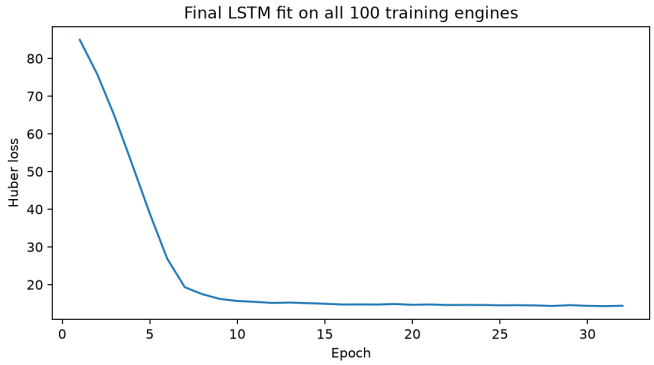


Fig. 9. Training loss for the final 32-epoch refit on all 100 training engines. This refit has no validation set; epoch count was selected beforehand.

TABLE VI
Official FD001 test set: final-cycle predictions, uncapped true RUL

Model	Validation RMSE	Test RMSE [95% CI]	Test MAE	NASA score
Phase 1 XGBoost	10.794	10.754 [9.1, 12.4]	8.085	161.109
Phase 2 LSTM	8.359	14.857 [12.4, 17.4]	10.559	309.965

VIII. Conclusion

On C-MAPSS FD001, the leakage-free engineered-feature XGBoost remains the strongest evaluated model, with official-test RMSE 10.754, MAE 8.085, and NASA score 161.109. The Phase 2 LSTM uses a leakage-safe engine split and sequence pipeline, but its official-test RMSE is 14.857, MAE 10.559, and NASA score 309.965. Although it scored better on the single validation holdout, that advantage did not generalize. This negative result is the central finding of the comparison: learned temporal representations did not improve this benchmark under the implemented data and protocol. Future work should use repeated engine-level validation and a new untouched evaluation population before considering model changes or a hybrid.

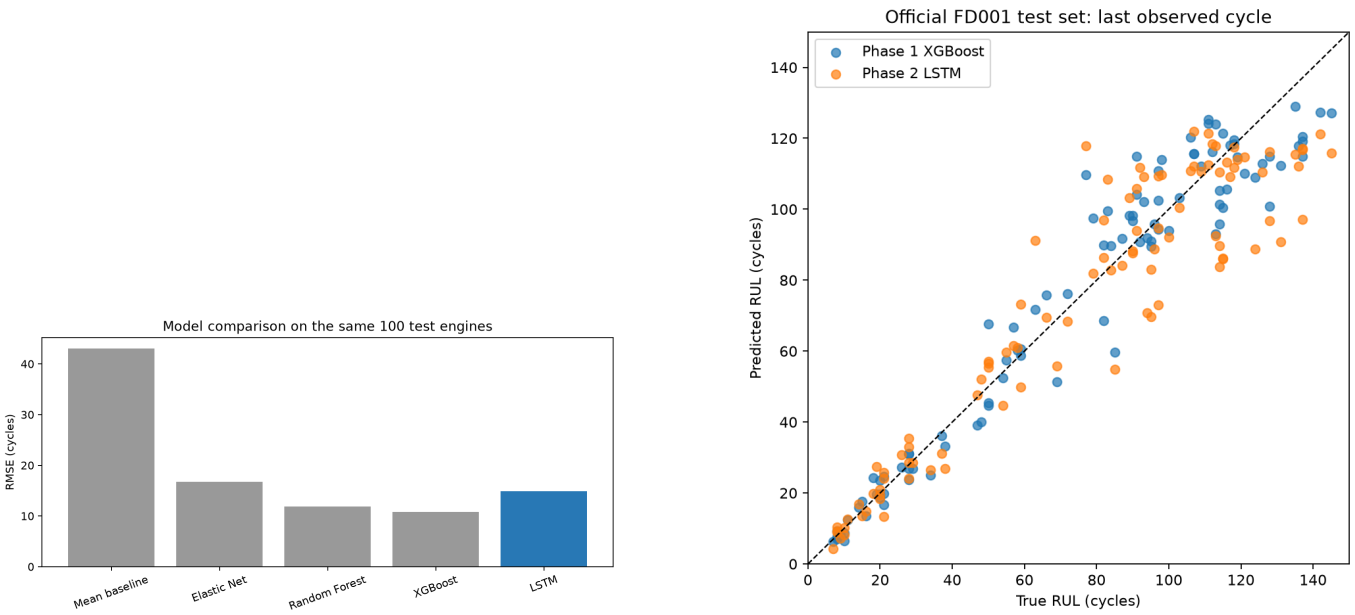


Fig. 10. Left: official-test metric comparison. Right: predictions at the last observed cycle for all 100 test engines; the dashed line is ideal prediction.

TABLE VII
Official test RMSE by stage, final cycle per engine

Stage	n	XGBoost	LSTM
Early (RUL $>$ 125)	11	17.594	26.306
Mid (50 $<$ RUL $\leq$ 125)	56	11.498	15.690
Late (RUL $\leq$ 50)	33	4.789	4.521

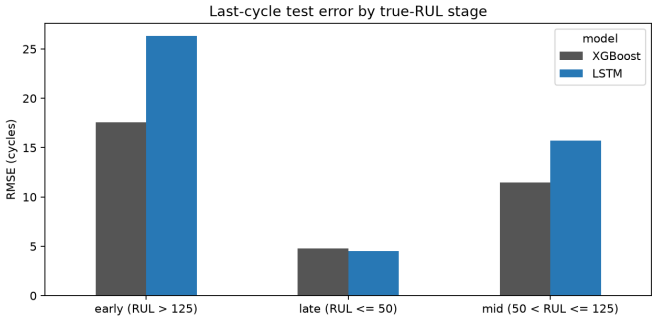


Fig. 11. Final-cycle test RMSE by degradation stage for XGBoost and LSTM; each engine contributes one observation.

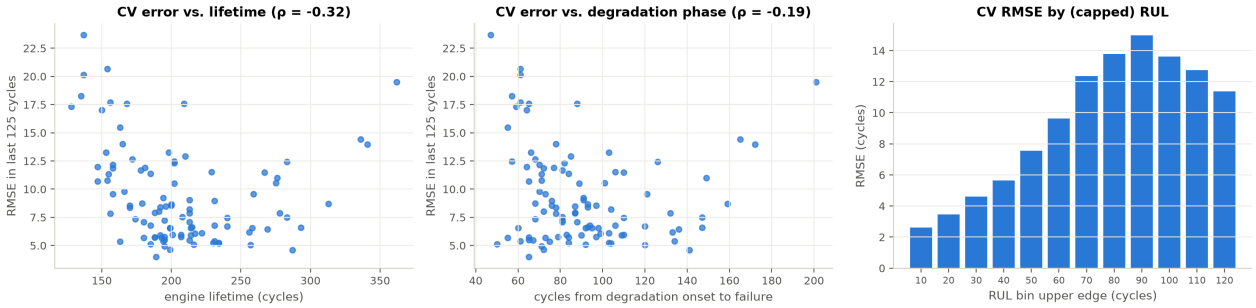


Fig. 12. Error analysis on out-of-fold CV predictions (XGBoost): per-engine error vs. lifetime and vs. length of the degradation phase, and RMSE by RUL bin.

Team Contributions

Member	Responsibility
Kushal Tyagi	Project lead; modelling, cross-validation and tuning; test evaluation and error analysis (notebooks 04–05, src/models.py, src/evaluation.py); documentation
Nishant Sharma	Exploratory analysis and feature engineering (notebooks 02–03, src/features.py)
Anant Lalwani	Literature review and report; data loading and quality checks (notebook 01)
Dev Tyagi	Data set sourcing and documentation (data/, download script)
Vansh Khod	Data cleaning and validation (data-integrity tests)

Code and notebooks: https://github.com/WhiteHacker000/Remaining-Useful-Life-Prediction-for-Industrial-Engines

References

[1] Y. Lei, N. Li, L. Guo, N. Li, T. Yan, and J. Lin, "Machinery health prognostics: A systematic review from data acquisition to RUL prediction," Mechanical Systems and Signal Processing, vol. 104, pp. 799–834, 2018.

[2] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in Proc. Int. Conf. Prognostics and Health Management (PHM), Denver, CO, USA, 2008, pp. 1–9.

[3] A. Saxena and K. Goebel, "Turbofan engine degradation simulation data set," NASA Ames Prognostics Data Repository, NASA Ames Research Center, Moffett Field, CA, 2008.

[4] T. Wang, J. Yu, D. Siegel, and J. Lee, "A similarity-based prognostics approach for remaining useful life estimation of engineered systems," in Proc. Int. Conf. Prognostics and Health Management (PHM), 2008, pp. 1–6.

[5] E. Ramasso and A. Saxena, "Performance benchmarking and analysis of prognostic methods for CMAPSS datasets," International Journal of Prognostics and Health Management, vol. 5, no. 2, 2014.

[6] F. O. Heimes, "Recurrent neural networks for remaining useful life estimation," in Proc. Int. Conf. Prognostics and Health Management (PHM), 2008, pp. 1–6.

[7] J. Coble and J. W. Hines, "Identifying optimal prognostic parameters from data: A genetic algorithms approach," in Proc. Annual Conf. Prognostics and Health Management Society, 2009.

[8] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[9] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," The Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.

[10] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794.

[11] H. Zou and T. Hastie, "Regularization and variable selection via the elastic net," Journal of the Royal Statistical Society: Series B, vol. 67, no. 2, pp. 301–320, 2005.

[12] G. Sateesh Babu, P. Zhao, and X.-L. Li, "Deep convolutional neural network based regression approach for estimation of remaining useful life," in Database Systems for Advanced Applications (DASFAA), ser. Lecture Notes in Computer Science, vol. 9642. Springer, 2016, pp. 214–228.

[13] S. Zheng, K. Ristovski, A. Farahat, and C. Gupta, "Long short-term memory network for remaining useful life estimation," in Proc. IEEE Int. Conf. Prognostics and Health Management (ICPHM), 2017, pp. 88–95.

[14] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[15] X. Li, Q. Ding, and J.-Q. Sun, "Remaining useful life estimation in prognostics using deep convolution neural networks," Reliability Engineering & System Safety, vol. 172, pp. 1–11, 2018.

[16] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," Journal of Machine Learning Research, vol. 13, pp. 281–305, 2012.

[17] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 4765–4774.